

Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования

Кафедра программного обеспечения информационных технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К курсовому проекту

на тему

ВЕБ-ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ

ИГРУ «КРЕСТИКИ-НОЛИКИ»

БГУИР КП 6-05-0612-01 001 ПЗ

Студент

Абакумов Г. Е.

Руководитель

Болтак С. В.

Минск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Сравнительный анализ существующих решений.....	6
1.2 Постановка задачи	7
2 ПРОЕКТИРОВАНИЕ ПРОГРАММЫ	8
2.1 Проектирование структуры программы	8
2.2 Проектирование системы сетевого взаимодействия.....	9
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	11
3.1 Описание разработанных модулей.....	11
3.2 Описание сетевого модуля.....	12
4 ТЕСТИРОВАНИЕ	14
5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	16
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	19
ПРИЛОЖЕНИЕ А. Исходный код программы	20
ПРИЛОЖЕНИЕ Б. Схема алгоритма работы системы	26

ВВЕДЕНИЕ

В современном мире сетевые игры занимают важное место в индустрии программного обеспечения. Многопользовательские приложения реального времени требуют продуманной архитектуры и надёжного механизма передачи данных между клиентами и сервером. Разработка подобных систем позволяет на практике освоить ключевые технологии веб-программирования: протокол WebSocket, асинхронную обработку событий, управление состоянием игры на сервере.

Целью настоящей курсовой проекта является проектирование и программная реализация веб-приложения, реализующего сетевую игру «Крестики-нолики» для двух игроков в режиме реального времени. Игра предполагает одновременное подключение двух участников через браузер, обмен ходами по сети и отображение актуального состояния доски каждому из них.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и рассмотреть существующие аналоги;
- спроектировать структуру программы и систему сетевого взаимодействия;
- реализовать серверную часть на языке Python с использованием фреймворка FastAPI и протокола WebSocket;
- реализовать клиентскую часть в виде HTML/JavaScript приложения;
- провести тестирование разработанного приложения;
- составить руководство пользователя.

Разработанное приложение представляет собой полноценную клиент-серверную систему, в которой сервер управляет игровыми комнатами и логикой игры, а клиент обеспечивает интерактивный пользовательский интерфейс. Технический стек включает Python 3.11, FastAPI, Uvicorn, а также HTML5, CSS3 и JavaScript на стороне клиента.

Практическая ценность работы заключается в освоении технологии WebSocket для организации двусторонней связи в реальном времени, изучении принципов асинхронного программирования на Python, а также приобретении навыков разработки интерактивных многопользовательских веб-приложений.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Сравнительный анализ существующих решений

Сетевые игры типа «крестики-нолики» широко представлены в сети Интернет. Большинство существующих реализаций можно разделить на несколько категорий по используемым технологиям и архитектурным решениям.

Первая категория – классические реализации на основе технологии Ajax (Asynchronous JavaScript and XML). В таких приложениях клиент периодически опрашивает сервер (polling) на предмет изменений состояния игры. Данный подход прост в реализации, однако обладает рядом существенных недостатков: значительная задержка обновления состояния (зависит от интервала опроса), избыточная нагрузка на сервер из-за множества холостых запросов, неэффективное использование сетевых ресурсов.

Вторая категория – приложения, основанные на технологии Server-Sent Events (SSE). SSE обеспечивает однонаправленный поток событий от сервера к клиенту. Это позволяет серверу мгновенно уведомлять клиентов об изменениях, однако для отправки данных от клиента к серверу по-прежнему используются обычные HTTP-запросы, что делает взаимодействие асимметричным.

Третья и наиболее современная категория – приложения на основе протокола WebSocket. WebSocket обеспечивает полнодуплексный канал связи поверх одного TCP-соединения. После установки соединения (handshake) обе стороны могут отправлять данные в любой момент без дополнительных заголовков HTTP, что минимизирует задержки и накладные расходы.

Анализ популярных онлайн-реализаций игры «крестики-нолики» (Minimax.io, Gametable.org, PlayingCards.io) показывает следующее:

- большинство современных реализаций используют WebSocket или Socket.IO (надстройка над WebSocket для Node.js);
- серверная логика, как правило, хранит состояние игровой сессии в оперативной памяти сервера;
- система комнат (rooms) является стандартным паттерном организации игровых сессий;
- для масштабируемости используются очереди сообщений (Redis Pub/Sub) при развёртывании на нескольких серверах.

В сравнении с библиотекой Socket.IO, которая предоставляет готовые механизмы комнат, автоматического переключения и фолбека на polling, чистая реализация на WebSocket через FastAPI даёт более полное понимание протокола и меньшие накладные расходы для небольших проектов.

Таким образом, для разрабатываемого приложения выбран стек на основе нативного WebSocket, что обеспечивает минимальные задержки, простоту отладки и отсутствие зависимости от сторонних библиотек реального времени.

1.2 Постановка задачи

В рамках данной курсового проекта необходимо разработать веб-приложение, реализующее сетевую игру «Крестики-нолики» для двух игроков. Приложение должно обеспечивать следующий функционал:

Серверная часть:

- управление игровыми комнатами: создание новой комнаты с генерацией уникального идентификатора, подключение к существующей комнате по идентификатору;
- хранение состояния игровой доски (9 ячеек) и текущего хода на стороне сервера;
- проверка условий завершения игры: победа одного из игроков или ничья;
- обработка отключения игрока с корректным сбросом состояния комнаты;
- рассылка актуального состояния игры всем участникам комнаты после каждого хода;
- поддержка одновременно нескольких независимых игровых комнат.

Клиентская часть:

- пользовательский интерфейс для создания или присоединения к игровой комнате;
- отображение игровой доски с возможностью совершения ходов;
- визуальная индикация текущего состояния игры (чей ход, победитель, ничья);
- обновление состояния доски в реальном времени без перезагрузки страницы;
- корректная обработка ошибок сетевого взаимодействия.

Технические ограничения и требования:

- язык программирования серверной части – Python 3.11 и выше;
- фреймворк – FastAPI с ASGI-сервером Uvicorn;
- протокол обмена данными – WebSocket (RFC 6455);
- формат сообщений – JSON;
- клиентская часть – HTML5, CSS3, JavaScript (без сторонних фреймворков);
- приложение должно корректно работать в современных браузерах (Chrome, Firefox, Edge).

2 ПРОЕКТИРОВАНИЕ ПРОГРАММЫ

2.1 Проектирование структуры программы

Разрабатываемое приложение построено по классической клиент-серверной архитектуре. Сервер является единственным источником истины (Single Source of Truth) – он хранит всё игровое состояние и рассылает его клиентам. Клиент лишь отображает полученное состояние и отправляет действия пользователя.

Серверная часть состоит из следующих компонентов:

- модуль `main.py` – точка входа приложения, содержит определение маршрутов FastAPI, логику обработки WebSocket-соединений и игровую логику;
- класс `RoomManager` – менеджер игровых комнат, обеспечивающий создание, поиск и удаление комнат с использованием `asyncio.Lock` для потокобезопасности;
- класс `Room` – представление игровой комнаты, хранящее список игроков, состояние доски, текущий ход и статус игры;
- класс `Player` – представление подключённого игрока, содержащее его идентификатор, символ (X или O) и WebSocket-соединение;
- функция `check_winner` – проверка условий победы по всем восьми возможным линиям.

Клиентская часть представлена единственным файлом `index.html`, который включает:

- HTML-разметку интерфейса игровой доски и информационных панелей;
- CSS-стили для визуального оформления;
- JavaScript-код для управления WebSocket-соединением и обновления DOM.

Взаимодействие между компонентами осуществляется по следующей схеме: клиент устанавливает WebSocket-соединение с сервером, отправляет сообщение типа `join` с указанием действия (`create` или `join`) и при необходимости идентификатором комнаты. Сервер назначает игроку символ, формирует снимок состояния игры и рассылает его всем участникам. При совершении хода клиент отправляет сообщение типа `move` с номером ячейки, сервер валидирует ход, обновляет состояние и рассылает обновлённый снимок.

Жизненный цикл комнаты:

- создание – первый игрок подключается с `action: "create"`, сервер генерирует уникальный идентификатор и назначает символ X;
- ожидание – комната находится в статусе `waiting` до подключения второго игрока;
- игра – при подключении второго игрока (символ O) статус меняется на `playing`, ход предоставляется игроку X;

- завершение – при победе одного из игроков или ничьей статус меняется на finished;
- сброс – при отключении игрока во время игры состояние доски сбрасывается, комната возвращается в статус waiting.

2.2 Проектирование системы сетевого взаимодействия

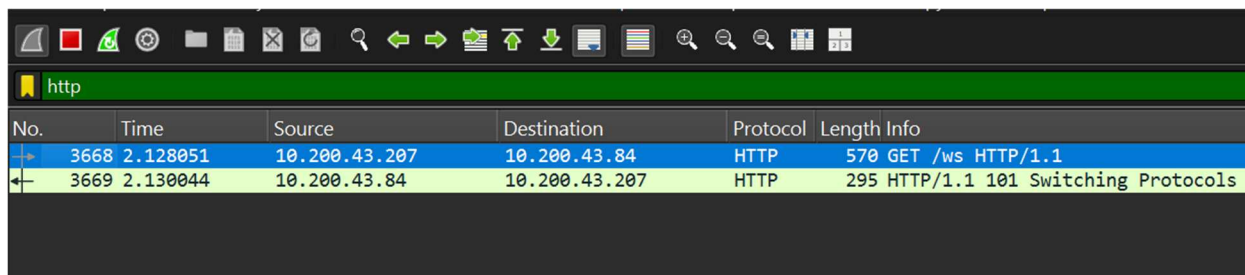
Система сетевого взаимодействия является ключевым компонентом разрабатываемого приложения. В её основе лежит протокол WebSocket, определённый в стандарте RFC 6455.

1 Протокол WebSocket и его особенности

WebSocket – протокол полнодуплексной связи поверх TCP, разработанный для использования в веб-браузерах и веб-серверах. Соединение инициируется клиентом через стандартный HTTP-запрос с заголовком Upgrade:

```
GET /ws HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Сервер отвечает кодом 101 Switching Protocols, после чего TCP-соединение переходит в режим WebSocket. Все последующие сообщения передаются во фреймах WebSocket, не имеющих HTTP-заголовков, что значительно снижает накладные расходы по сравнению с обычными HTTP-запросами. На рисунке 2.1 показана информация из Wireshark.



No.	Time	Source	Destination	Protocol	Length	Info
3668	2.128051	10.200.43.207	10.200.43.84	HTTP	570	GET /ws HTTP/1.1
3669	2.130044	10.200.43.84	10.200.43.207	HTTP	295	HTTP/1.1 101 Switching Protocols

Рисунок 2.1 – Скриншот Wireshark(WebSocket соединение)

Ключевые характеристики протокола WebSocket применительно к данному проекту:

- низкая задержка – отсутствие HTTP-заголовков при каждом сообщении сокращает overhead до нескольких байт на фрейм;
- полный дуплекс – сервер может отправить данные клиенту в любой момент без запроса с его стороны;
- постоянное соединение – единожды установленное соединение сохраняется на время сессии;

– поддержка текстовых и бинарных фреймов – в данном проекте используются текстовые фреймы с JSON-данными.

2 Протокол обмена сообщениями

Для обмена данными между клиентом и сервером разработан текстовый протокол на основе JSON. Каждое сообщение содержит поле `type`, определяющее тип сообщения, и дополнительные поля с данными.

Сообщения от клиента к серверу:

– `join` – подключение к игре. Поля: `type` ("join"), `action` ("create" или "join"), `room_id` (только для `action=join`), `player_id` (необязательно);

– `move` – совершение хода. Поля: `type` ("move"), `cell` (целое число от 0 до 8);

– `ping` – проверка соединения. Поля: `type` ("ping").

Сообщения от сервера к клиенту:

– `joined` – подтверждение подключения. Поля: `type` ("joined"), `room_id`, `player_id`, `symbol` ("X" или "O");

– `state` – снимок состояния игры. Поля: `type` ("state"), `room_id`, `status` ("waiting"/"playing"/"finished"), `board` (массив из 9 строк), `turn` ("X" или "O"), `winner` ("X", "O" или null), `players` ({X: id, O: id});

– `pong` – ответ на `ping`. Поля: `type` ("pong");

– `error` – сообщение об ошибке. Поля: `type` ("error"), `message`.

3 Асинхронная обработка соединений

FastAPI использует ASGI (Asynchronous Server Gateway Interface) – стандарт для асинхронных Python веб-серверов. Каждое WebSocket-соединение обрабатывается в отдельной асинхронной корутине, что позволяет серверу параллельно обслуживать множество соединений в рамках одного процесса без создания отдельного потока для каждого клиента.

Для предотвращения состояния гонки (race condition) при одновременном доступе нескольких корутин к данным игровой комнаты используется `asyncio.Lock`. Блокировка устанавливается перед чтением или изменением состояния комнаты и снимается после завершения операции.

Широковещательная рассылка (broadcast) реализована с использованием `asyncio.gather`, что позволяет отправлять сообщения всем участникам комнаты параллельно, не дожидаясь завершения отправки каждому из них по отдельности.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Описание разработанных модулей

Серверная часть приложения реализована в одном файле `main.py`, содержащем все необходимые классы и функции. Рассмотрим основные компоненты реализации.

1 Функция `check_winner`

Функция проверяет наличие победителя на игровой доске. Доска представлена списком из 9 строк, где пустая строка обозначает свободную ячейку, а «X» или «O» – занятую. Функция перебирает восемь возможных выигрышных комбинаций (три горизонтали, три вертикали, две диагонали) и возвращает символ победителя или `None`.

```
def check_winner(board: list[str]) -> PlayerSymbol | None:
    wins = [(0,1,2), (3,4,5), (6,7,8),
            (0,3,6), (1,4,7), (2,5,8),
            (0,4,8), (2,4,6)]
    for a, b, c in wins:
        if board[a] != '' and board[a] == board[b] == board[c]:
            return board[a]
    return None
```

2 Класс `Player`

Датакласс `Player` хранит идентификатор игрока, его символ (X или O) и ссылку на `WebSocket`-объект для отправки сообщений. Хранение `WebSocket` в объекте игрока позволяет легко рассылать сообщения конкретному участнику.

3 Класс `Room`

Датакласс `Room` представляет игровую комнату и содержит: уникальный идентификатор комнаты, словарь игроков (ключ – символ X или O), состояние доски, символ игрока, чей ход, статус игры, символ победителя и `asyncio.Lock` для синхронизации доступа.

Метод `snapshot()` формирует словарь, описывающий текущее состояние игры и пригодный для сериализации в JSON. Метод намеренно не включает объекты `WebSocket`, так как они не сериализуемы.

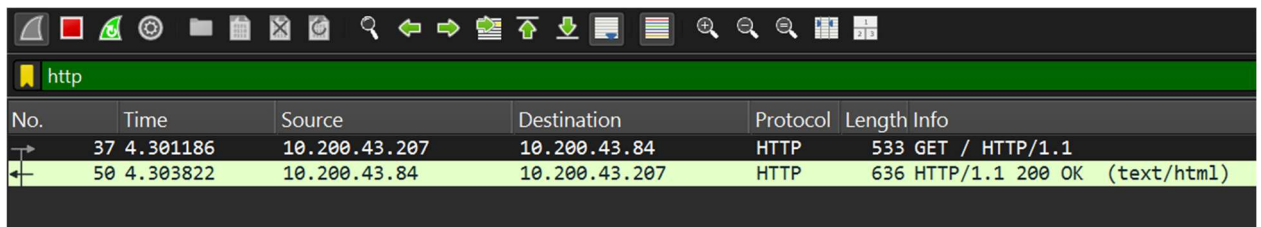
4 Класс `RoomManager`

Синглтон-объект `rooms` управляет всеми активными комнатами. Для защиты словаря комнат от конкурентного доступа используется глобальная блокировка `_global_lock`. Метод `create_room` генерирует уникальный идентификатор с помощью `secrets.token_urlsafe(6)` – функции из стандартной библиотеки, обеспечивающей криптографически стойкую случайность. Метод `delete_room_if_empty` удаляет комнату, если в ней не осталось игроков, освобождая память.

5 Маршрут HTTP GET /

Возвращает содержимое файла `static/index.html` как HTML-ответ. FastAPI также монтирует директорию `static/` для раздачи статических файлов (CSS, JavaScript).

На рисунке 3.1 показана информация из Wireshark.



No.	Time	Source	Destination	Protocol	Length	Info
37	4.301186	10.200.43.207	10.200.43.84	HTTP	533	GET / HTTP/1.1
50	4.303822	10.200.43.84	10.200.43.207	HTTP	636	HTTP/1.1 200 OK (text/html)

Рисунок 3.1 – Скриншот WireShark(передача static/index.html)

6 Вспомогательные функции

Функция `safe_send` обёртывает отправку JSON-сообщения через WebSocket в блок `try/except`. Это необходимо, так как соединение может быть разорвано в любой момент, и попытка отправки в закрытое соединение вызывает исключение.

Функция `broadcast` использует `asyncio.gather` для параллельной рассылки сообщений всем игрокам комнаты.

Функция `assign_symbol` определяет, какой символ присвоить новому игроку: X – если первый слот свободен, O – если свободен второй, None – если комната заполнена.

3.2 Описание сетевого модуля

Сетевой модуль реализован в виде асинхронного обработчика WebSocket-эндпоинта `/ws`. Рассмотрим его работу поэтапно.

1 Установка соединения и первое сообщение

При подключении клиента сервер принимает соединение вызовом `await ws.accept()`. Затем ожидает первое сообщение, которое должно быть JSON-объектом с `type: "join"`. Это сообщение содержит информацию о желаемом действии (создать или присоединиться к комнате). Если формат сообщения некорректен, соединение закрывается с сообщением об ошибке.

2 Обработка подключения к комнате

В зависимости от поля `action` первого сообщения:

- `action: "create"` – создаётся новая комната через `rooms.create_room()`;
- `action: "join"` – выполняется поиск существующей комнаты по `room_id` через `rooms.get_room()`; если комната не найдена, отправляется сообщение об ошибке.

Далее под блокировкой комнаты выполняется назначение символа новому игроку. Если комната заполнена (оба символа заняты), соединение закрывается. При успешном подключении второго игрока статус комнаты меняется с `waiting` на `playing`.

После успешного подключения сервер отправляет игроку персональное сообщение `joined` с его данными и рассылает всем участникам снимок состояния игры.

3 Основной цикл обработки сообщений

После подключения обработчик входит в бесконечный цикл ожидания сообщений. Поддерживаются следующие типы:

Обработка хода (type: "move"):

- валидация номера ячейки (от 0 до 8);
- проверка статуса игры (должен быть playing);
- проверка очерёдности хода (symbol должен совпадать с room.turn);
- проверка занятости ячейки;
- запись символа в доску;
- проверка условий завершения игры (победа или ничья);
- обновление текущего хода;
- широковещательная рассылка обновлённого состояния.

Все операции с состоянием комнаты выполняются под блокировкой room.lock для предотвращения гонки данных. Широковещательная рассылка выполняется вне блокировки, так как отправка сообщений может занять значительное время при медленных соединениях, и удержание блокировки на это время блокировало бы других игроков.

4 Обработка отключения

При разрыве соединения возникает исключение WebSocketDisconnect. Блок finally гарантированно выполняется в любом случае – как при нормальном завершении, так и при исключении. В нём выполняется:

– удаление игрока из словаря участников комнаты (только если WebSocket-объект совпадает, чтобы избежать удаления нового игрока, переиспользовавшего слот);

- сброс состояния игры, если она была в процессе;
- рассылка обновлённого состояния оставшимся игрокам;
- удаление пустой комнаты.

5 Безопасность и обработка ошибок

Функция safe_send используется везде, где отправляется сообщение конкретному клиенту, а не через broadcast. Это позволяет безопасно отправить сообщение об ошибке даже если соединение уже в процессе закрытия. Все исключения, не являющиеся WebSocketDisconnect, перехватываются и отправляются клиенту в виде error-сообщения.

6 Использование secrets вместо random

Для генерации идентификаторов комнат и игроков используется модуль secrets стандартной библиотеки Python. В отличие от модуля random, secrets использует системный генератор случайных чисел (os.urandom), который обеспечивает криптографически стойкую случайность. Это исключает возможность предсказания идентификатора комнаты злоумышленником.

4 ТЕСТИРОВАНИЕ

Тестирование разработанного приложения проводилось на нескольких уровнях: модульное тестирование игровой логики, интеграционное тестирование WebSocket-взаимодействия и функциональное тестирование пользовательского сценария.

1 Модульное тестирование функции `check_winner`

Функция `check_winner` проверялась на всех восьми возможных выигрышных комбинациях для обоих символов, а также на случаях незавершённой игры и ничьей. Тестовые случаи:

- победа X по горизонтали: доска `["X","X","X","","","","",""]` – результат `"X"`;
- победа O по вертикали: доска `["","O","","","O","","","O",""]` – результат `"O"`;
- победа X по диагонали: доска `["X","","","","X","","","X"]` – результат `"X"`;
- незавершённая игра: доска `["X","O","X","","","","",""]` – результат `None`;
- ничья: полная доска без победителя – результат `None`.

Все тестовые случаи дали ожидаемые результаты.

2 Тестирование WebSocket-соединения

Для тестирования сетевого взаимодействия использовался инструмент `pytest` совместно с `httpx` и библиотекой `websockets`. Тестировались следующие сценарии:

- корректное создание комнаты: клиент отправляет `join` с `action="create"`, получает сообщение `joined` с валидным `room_id` и `symbol="X"`;
- корректное подключение к комнате: второй клиент отправляет `join` с `action="join"` и `room_id`, получает `symbol="O"`, оба клиента получают `state` с `status="playing"`;
- подключение к несуществующей комнате: клиент получает `error` с соответствующим сообщением;
- попытка подключения третьего клиента к заполненной комнате: получает `error "Room is full"`;
- совершение хода не в свою очередь: получает `error "Not your turn"`;
- совершение хода в занятую ячейку: получает `error "Cell already taken"`.

3 Функциональное тестирование в браузере

Функциональное тестирование проводилось в браузерах Google Chrome и Mozilla Firefox на операционной системе Windows 11. Тестировались следующие пользовательские сценарии:

- полный игровой цикл: создание комнаты, подключение второго игрока, проведение партии до победы – работает корректно в обоих браузерах;
- игра до ничьей: все 9 ячеек заняты без победителя – статус `correctly` отображается как ничья;

- отключение игрока в середине партии: оставшийся игрок получает обновление состояния, комната сбрасывается – работает корректно;
- обновление страницы во время игры: игрок переподключается и может создать новую игру – работает корректно;
- одновременное проведение нескольких партий в разных вкладках браузера – работает корректно, комнаты независимы.

4 Нагрузочное тестирование

Было проведено ограниченное нагрузочное тестирование с использованием скрипта на Python, создающего 50 одновременных WebSocket-соединений (25 игровых пар). На рисунке 4.1 и 4.2 показаны результаты тестирования.

```

===== RESULT =====
Pairs: 25
Clients: 50
Successful connections: 50
Failed: 0
Completed games: 25
Time: 4.94s
=====
Process finished with exit code 0

```

Рисунок 4.1 – результаты теста

302	9.200134	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
304	9.201730	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
306	9.202054	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
308	9.202333	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
310	9.202388	127.0.0.1	127.0.0.1	HTTP	273 HTTP/1.1 101 Switching Protocols
314	9.206436	127.0.0.1	127.0.0.1	HTTP	273 HTTP/1.1 101 Switching Protocols
318	9.208316	127.0.0.1	127.0.0.1	HTTP	273 HTTP/1.1 101 Switching Protocols
322	9.209098	127.0.0.1	127.0.0.1	HTTP	273 HTTP/1.1 101 Switching Protocols
459	9.235448	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
461	9.235802	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
463	9.236206	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
465	9.236523	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
467	9.236810	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
469	9.237088	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
473	9.237855	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
475	9.238123	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
477	9.238675	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
479	9.238952	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
481	9.239264	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
483	9.239559	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
485	9.239816	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
487	9.240099	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
489	9.240412	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
491	9.240687	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
493	9.240935	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
495	9.241236	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
497	9.241584	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
499	9.241853	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
501	9.242137	127.0.0.1	127.0.0.1	HTTP	310 GET /ws HTTP/1.1
507	9.257914	127.0.0.1	127.0.0.1	HTTP	273 HTTP/1.1 101 Switching Protocols
511	9.268234	127.0.0.1	127.0.0.1	HTTP	273 HTTP/1.1 101 Switching Protocols
513	9.269019	127.0.0.1	127.0.0.1	HTTP	273 HTTP/1.1 101 Switching Protocols

Рисунок 4.2 – результаты теста Wireshark

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Данный раздел описывает процедуру установки, запуска и использования веб-приложения «Крестики-нолики».

1 Системные требования

Для работы серверной части приложения необходимо:

- операционная система: Windows 10/11, macOS 12+, Linux (Ubuntu 20.04+);

- Python версии 3.11 или выше;

- установленный пакетный менеджер pip;

- свободный сетевой порт 8000 (или любой другой по желанию).

Для работы клиентской части необходим современный браузер с поддержкой WebSocket:

- Google Chrome;

- Mozilla Firefox;

- Microsoft Edge.

2 Установка зависимостей

Для установки необходимых Python-библиотек выполните следующую команду в терминале из директории проекта:

```
pip install fastapi uvicorn
```

Альтернативно, если в проекте присутствует файл requirements.txt:

```
pip install -r requirements.txt
```

3 Запуск сервера

Для запуска приложения выполните команду в директории, содержащей файл main.py:

```
uvicorn main:app --reload
```

После успешного запуска в терминале отобразится сообщение:

```
INFO:      Uvicorn running on http://127.0.0.1:8000
```

Флаг --reload обеспечивает автоматическую перезагрузку сервера при изменении исходного кода и рекомендуется только для разработки. Для продакшн-окружения используйте:

```
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 1
```

Обратите внимание: из-за использования глобального объекта rooms в памяти процесса, приложение должно запускаться с одним воркером (--workers 1). При необходимости горизонтального масштабирования потребуется вынести хранилище состояний в внешнюю БД (например, Redis).

4 Использование приложения

Шаг 1. Откройте браузер и перейдите по адресу <http://127.0.0.1:8000>. Отобразится главная страница приложения с двумя кнопками: «Создать игру» и «Присоединиться к игре».

Шаг 2. Первый игрок нажимает кнопку «Создать игру». Сервер создаёт новую комнату и возвращает её идентификатор, который отображается на экране. Первый игрок получает символ X.

Шаг 3. Первый игрок сообщает идентификатор комнаты второму игроку (например, скопировав его из адресной строки или текстового поля). Второй

игрок открывает приложение в своём браузере, нажимает «Присоединиться к игре», вводит полученный идентификатор и нажимает «Войти». Второй игрок получает символ О.

Шаг 4. После подключения обоих игроков игра начинается автоматически. На экране обоих игроков отображается игровая доска 3×3 и надпись с указанием, чей ход. Первый ход предоставляется игроку Х.

Шаг 5. Игрок, чей ход, кликает на свободную ячейку доски. Его символ отображается в ячейке, а ход передаётся сопернику. Доска обновляется у обоих игроков одновременно.

Шаг 6. Игра продолжается до тех пор, пока один из игроков не выстроит три своих символа в линию по горизонтали, вертикали или диагонали, либо пока все 9 ячеек не будут заняты (ничья). По завершении игры на экране обоих игроков отображается результат.

Шаг 7. После завершения партии игроки могут начать новую игру, нажав соответствующую кнопку, или вернуться на главную страницу.

5 Возможные проблемы и их решение

Не удаётся подключиться к серверу: убедитесь, что сервер запущен и порт 8000 не заблокирован брандмауэром. Проверьте корректность адреса в браузере.

Соединение обрывается во время игры: в большинстве случаев это связано с нестабильностью сети. Перезагрузите страницу и создайте новую игровую комнату.

Второй игрок не может подключиться: убедитесь, что идентификатор комнаты введён корректно (с учётом регистра). Идентификатор чувствителен к регистру символов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было спроектировано и реализовано веб-приложение, реализующее сетевую игру «Крестики-нолики» для двух игроков в режиме реального времени.

В процессе работы были решены все поставленные задачи:

- проведён анализ предметной области и сравнительный анализ существующих технологий для организации сетевого взаимодействия в реальном времени (Ajax polling, Server-Sent Events, WebSocket);
- разработана архитектура клиент-серверного приложения с чётким разделением ответственности: сервер хранит состояние и реализует игровую логику, клиент отображает состояние и передаёт действия пользователя;
- спроектирован и реализован протокол обмена сообщениями на основе JSON поверх WebSocket;
- реализована серверная часть на Python с использованием FastAPI и asyncio, обеспечивающая управление несколькими одновременными игровыми комнатами с потокобезопасным доступом к общим данным;
- реализован сетевой модуль с полным циклом управления WebSocket-соединением: установка, обработка сообщений, корректное завершение;
- проведено тестирование на уровнях модульных тестов, интеграционного тестирования и функционального тестирования в браузере;
- составлено руководство пользователя, описывающее установку, запуск и использование приложения.

Разработанное приложение полностью соответствует предъявленным требованиям: поддерживает одновременное проведение нескольких партий, корректно обрабатывает отключение игроков, обеспечивает синхронизацию состояния доски у обоих участников в режиме реального времени.

В ходе работы были практически освоены: протокол WebSocket и его отличия от HTTP; принципы асинхронного программирования на Python с использованием asyncio; разработка REST и WebSocket API с помощью FastAPI; методы обеспечения потокобезопасности в асинхронных приложениях.

Возможными направлениями дальнейшего развития проекта являются: добавление персистентного хранилища (PostgreSQL, Redis) для хранения истории партий и статистики игроков; реализация аутентификации и рейтинговой системы; поддержка игры с компьютером на основе алгоритма Minimax; развёртывание на облачном сервере с поддержкой HTTPS/WSS.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- [1] RFC 6455: The WebSocket Protocol [Электронный ресурс]. – Режим доступа: <https://datatracker.ietf.org/doc/html/rfc6455>.
- [2] FastAPI Documentation [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com>.
- [3] Python asyncio – Asynchronous I/O [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/asyncio.html>.
- [4] Лукасевич, Р. Python. Асинхронное программирование / Р. Лукасевич. – М.: ДМК Пресс, 2023. – 432 с.
- [5] Гриффитс, И. Программирование на Python. Полное руководство / И. Гриффитс. – СПб.: Питер, 2022. – 624 с.
- [6] MDN Web Docs: WebSocket API [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>.
- [7] Uvicorn Documentation [Электронный ресурс]. – Режим доступа: <https://www.uvicorn.org>.
- [8] Лутц, М. Программирование на Python. В 2 т. / М. Лутц. – СПб.: Символ-Плюс, 2021. – Т. 1. – 992 с.
- [9] Python secrets module documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/secrets.html>.
- [10] ASGI Specification [Электронный ресурс]. – Режим доступа: <https://asgi.readthedocs.io/en/latest/>.

ПРИЛОЖЕНИЕ А. Исходный код программы

Содержимое файла «main.py»:

```
from __future__ import annotations

import asyncio
import secrets
from dataclasses import dataclass, field
from typing import Literal

from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles

PlayerSymbol = Literal["X", "O"]
GameStatus = Literal["waiting", "playing", "finished"]

def check_winner(board: list[str]) -> PlayerSymbol | None:
    wins = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6),
    ]
    for a, b, c in wins:
        if board[a] != "" and board[a] == board[b] == board[c]:
            return board[a] # type: ignore[return-value]
    return None

@dataclass
class Player:
    player_id: str
    symbol: PlayerSymbol
    ws: WebSocket

@dataclass
```

```

class Room:
    room_id: str
    players: dict[PlayerSymbol, Player] = field(default_factory=dict)
    board: list[str] = field(default_factory=lambda: [""] * 9)
    turn: PlayerSymbol = "X"
    status: GameStatus = "waiting"
    winner: PlayerSymbol | None = None
    lock: asyncio.Lock = field(default_factory=asyncio.Lock)

    def snapshot(self) -> dict:
        return {
            "type": "state",
            "room_id": self.room_id,
            "status": self.status,
            "board": self.board,
            "turn": self.turn,
            "winner": self.winner,
            "players": {
                "X": self.players["X"].player_id if "X" in self.players else None,
                "O": self.players["O"].player_id if "O" in self.players else None,
            },
        }

```

```

class RoomManager:
    def __init__(self) -> None:
        self._rooms: dict[str, Room] = {}
        self._global_lock = asyncio.Lock()

    async def create_room(self) -> Room:
        async with self._global_lock:
            while True:
                rid = secrets.token_urlsafe(6)
                if rid not in self._rooms:
                    room = Room(room_id=rid)
                    self._rooms[rid] = room
                    return room

    async def get_room(self, room_id: str) -> Room | None:
        async with self._global_lock:
            return self._rooms.get(room_id)

    async def delete_room_if_empty(self, room_id: str) -> None:
        async with self._global_lock:

```

```

        room = self._rooms.get(room_id)
        if room is not None and not room.players:
            del self._rooms[room_id]

rooms = RoomManager()

app = FastAPI()

from pathlib import Path
import os

BASE_DIR = Path(__file__).resolve().parent.parent
app.mount("/static", StaticFiles(directory=BASE_DIR / "static"), name="static")

@app.get("/", response_class=HTMLResponse)
async def index() -> HTMLResponse:
    html_file_path = os.path.join(BASE_DIR, "static", "index.html")
    with open(html_file_path, "r", encoding="utf-8") as f:
        return HTMLResponse(f.read())

async def safe_send(ws: WebSocket, message: dict) -> None:
    try:
        await ws.send_json(message)
    except Exception:
        # Connection likely gone; ignore, cleanup happens elsewhere.
        pass

async def broadcast(room: Room, message: dict) -> None:
    await asyncio.gather(
        *[safe_send(p.ws, message) for p in room.players.values()],
        return_exceptions=True,
    )

def assign_symbol(room: Room) -> PlayerSymbol | None:
    if "X" not in room.players:
        return "X"
    if "O" not in room.players:
        return "O"
    return None

```

```

@app.websocket("/ws")
async def ws_endpoint(ws: WebSocket) -> None:
    await ws.accept()

    room: Room | None = None
    symbol: PlayerSymbol | None = None
    player_id: str | None = None

    try:
        first = await ws.receive_json()
        if not isinstance(first, dict) or first.get("type") != "join":
            await ws.send_json({"type": "error", "message": "First message must be
join"})
            await ws.close()
            return

        desired_room = first.get("room_id")
        action = first.get("action")
        player_id = str(first.get("player_id") or secrets.token_hex(4))

        if action == "create":
            room = await rooms.create_room()
        elif action == "join" and isinstance(desired_room, str):
            room = await rooms.get_room(desired_room)
            if room is None:
                await ws.send_json({"type": "error", "message": "Room not found"})
                await ws.close()
                return
        else:
            await ws.send_json({"type": "error", "message": "Invalid join payload"})
            await ws.close()
            return

        async with room.lock:
            symbol = assign_symbol(room)
            if symbol is None:
                await ws.send_json({"type": "error", "message": "Room is full"})
                await ws.close()
                return

            room.players[symbol] = Player(player_id=player_id, symbol=symbol,
ws=ws)

```

```

    if len(room.players) == 2 and room.status == "waiting":
        room.status = "playing"
        room.turn = "X"

    await ws.send_json(
        {
            "type": "joined",
            "room_id": room.room_id,
            "player_id": player_id,
            "symbol": symbol,
        }
    )
    await broadcast(room, room.snapshot())

    while True:
        msg = await ws.receive_json()
        if not isinstance(msg, dict):
            continue

        if msg.get("type") == "move":
            if room is None or symbol is None:
                continue

            cell = msg.get("cell")
            if not isinstance(cell, int) or cell < 0 or cell > 8:
                await safe_send(ws, {"type": "error", "message": "Invalid cell"})
                continue

            async with room.lock:
                if room.status != "playing":
                    await safe_send(ws, {"type": "error", "message": "Game not in
playing state"})
                    continue
                if room.turn != symbol:
                    await safe_send(ws, {"type": "error", "message": "Not your turn"})
                    continue
                if room.board[cell] != "":
                    await safe_send(ws, {"type": "error", "message": "Cell already
taken"})
                    continue

                room.board[cell] = symbol
                w = check_winner(room.board)
                if w is not None:

```



```

        room.status = "finished"
        room.winner = w
    elif all(v != "" for v in room.board):
        room.status = "finished"
        room.winner = None
    else:
        room.turn = "O" if room.turn == "X" else "X"

    await broadcast(room, room.snapshot())

elif msg.get("type") == "ping":
    await safe_send(ws, {"type": "pong"})

else:
    await safe_send(ws, {"type": "error", "message": "Unknown message
type"})

except WebSocketDisconnect:
    pass
except Exception as e:
    try:
        await ws.send_json({"type": "error", "message": f"Server error: {e}"})
    except Exception:
        pass
finally:
    if room is not None and symbol is not None:
        async with room.lock:
            # remove only if still same connection
            p = room.players.get(symbol)
            if p is not None and p.ws is ws:
                del room.players[symbol]
            # reset game if someone leaves mid-game
            if room.status == "playing":
                room.status = "waiting"
                room.board = ["" ] * 9
                room.turn = "X"
                room.winner = None
            await broadcast(room, room.snapshot())
            await rooms.delete_room_if_empty(room.room_id)(room.room_id)

```

ПРИЛОЖЕНИЕ Б. Схема алгоритма работы системы